

Candlestick.cpp

```
1 | #include "Candlestick.h"  
2 |  
3 | // Implementation is already in the header as it's a simple inline class  
4 |
```

Candlestick.h

```
1 //my code
2 #ifndef CANDLESTICK_H
3 #define CANDLESTICK_H
4
5 #include <string>
6 #include <iostream>
7
8 //candlestick class models OHLC data for a specific date
9 class Candlestick {
10 public:
11     //attributes representing the date and OHLC values
12     std::string date;
13     double open;
14     double high;
15     double low;
16     double close;
17
18     //constructor to initialize a candlestick with the given parameters
19     Candlestick(const std::string& date, double open, double high, double low,
20 double close)
21         : date(date), open(open), high(high), low(low), close(close) {}
22
23     //displaying the candlestick data in a simple format
24     void display() const {
25         //printing the OHCL values
26         std::cout << date << " "
27             << open << " "
28             << high << " "
29             << low << " "
30             << close << std::endl;
31     }
32 };
33 #endif
34 //my code
```

CandlestickPlotter.cpp

```
1 //used my teachers assistance in this code
2 #include "CandlestickPlotter.h"
3 #include <iostream>
4 #include <iomanip>
5 #include <cmath>
6 #include <tuple>
7 #include <string>
8 #include <vector>
9 #include <sstream>
10 #include <algorithm>
11
12 //using ANSI escape codes for colors
13 #define RED "\033[31m"
14 #define GREEN "\033[32m"
15 #define RESET "\033[0m"
16
17 //normalizing a value for plotting and mapping the given value to a scale for
graphical representation
18 int normalize(double value, double min, double max, int scale) {
19     double normalizedValue = (value - min) / (max - min) * scale;
20     //ensures values fit within the range
21     int result = std::round(normalizedValue);
22     return result;
23 }
24
25 //filtering data based on the users selected year range and also ensuring only
relevant years are included for processing
26 std::vector<std::vector<std::string>> filterDataByYearRange(
27     const std::vector<std::vector<std::string>>& data, int startYear, int endYear)
28 {
29     std::vector<std::vector<std::string>> filteredData;
30
31     for (const auto& row : data) {
32         //extractig the year from date
33         std::string yearString = row[0].substr(0, 4);
34         int year = std::stoi(yearString);
35         if (year >= startYear && year <= endYear) {
36             //adding rows that fit the year range
37             filteredData.push_back(row);
38         }
39     }
40
41     return filteredData;
42 }
43
44 //getting the minimum and maximum years from the dataset that are usefull for
determining available data boundaries
45 std::pair<int, int> getYearRange(const std::vector<std::vector<std::string>>&
data) {
46     int minYear = std::numeric_limits<int>::max();
47     int maxYear = std::numeric_limits<int>::min();
48
49     for (const auto& row : data) {
50         //extracting year from date
51         std::string yearString = row[0].substr(0, 4);
52         int year = std::stoi(yearString);
53         //tracking earliest and latest years
54         minYear = std::min(minYear, year);
```

```
54     maxYear = std::max(maxYear, year);
55 }
56
57 return {minYear, maxYear};
58 }
59
60 //plotting candlestick data for a given dataset
61 void CandlestickPlotter::plotCandlesticks(const std::vector<std::vector<
std::string>>& ohlcData) {
62     const int plotHeight = 100;
63     const int candlestickWidth = 10;
64     const double bufferFactor = 0.2;
65
66     if (ohlcData.empty()) {
67         //informing the user about empty dataset
68         std::cerr << "No data to plot!\n";
69         return;
70     }
71
72     //getting the available year range from the dataset
73     int minYear = std::numeric_limits<int>::max();
74     int maxYear = std::numeric_limits<int>::min();
75     for (const auto& row : ohlcData) {
76         //extracting year from the date
77         int year = std::stoi(row[0].substr(0, 4));
78         minYear = std::min(minYear, year);
79         maxYear = std::max(maxYear, year);
80     }
81
82     std::cout << "Available years: " << minYear << " to " << maxYear << "\n";
83
84     //asking user for year range input
85     int startYear, endYear;
86     std::cout << "Enter the starting year: ";
87     std::cin >> startYear;
88     std::cout << "Enter the ending year: ";
89     std::cin >> endYear;
90
91     //validating the input year range given by user
92     if (startYear < minYear || endYear > maxYear || startYear > endYear) {
93         std::cerr << "Invalid year range. Please select between " << minYear << "
and " << maxYear << ".\n";
94         return;
95     }
96
97     //filtering data by the selected year range
98     std::vector<std::vector<std::string>> filteredData;
99     for (const auto& row : ohlcData) {
100         //extracting year from the date
101         int year = std::stoi(row[0].substr(0, 4));
102         if (year >= startYear && year <= endYear) {
103             filteredData.push_back(row);
104         }
105     }
106
107     if (filteredData.empty()) {
108         std::cerr << "No data available for the selected year range!\n";
109         return;
110     }
111 }
```

```

112 // Display the OHLC table for the user selected years
113 std::cout << "\n===== OHLC Values of Candlesticks =====
=\\n";
114 std::cout << std::setw(12) << "Date" << std::setw(12) << "Open"
115 << std::setw(12) << "High" << std::setw(12) << "Low"
116 << std::setw(12) << "Close\\n";
117 for (const auto& row : filteredData) {
118     std::cout << std::setw(12) << row[0]
119 << std::setw(12) << std::fixed << std::setprecision(3) << row[1]
120 << std::setw(12) << row[2]
121 << std::setw(12) << row[3]
122 << std::setw(12) << row[4] << "\\n";
123 }
124 std::cout << "=====
\\n";

125
126 double globalHigh = -std::numeric_limits<double>::infinity();
127 double globalLow = std::numeric_limits<double>::infinity();
128
129 //finding global min and max for normalization
130 for (const auto& row : filteredData) {
131     double high = std::stod(row[2]);
132     double low = std::stod(row[3]);
133     globalHigh = std::max(globalHigh, high);
134     globalLow = std::min(globalLow, low);
135 }
136
137 //buffering the Y-axis range
138 double range = globalHigh - globalLow;
139 globalHigh += range * bufferFactor;
140 globalLow -= range * bufferFactor;
141
142 //precomputing the normalized positions for all candlesticks
143 std::vector<std::tuple<int, int, int, int>> normalizedCandlesticks;
144 for (const auto& row : filteredData) {
145     double open = std::stod(row[1]);
146     double high = std::stod(row[2]);
147     double low = std::stod(row[3]);
148     double close = std::stod(row[4]);
149
150     int highPos = normalize(high, globalLow, globalHigh, plotHeight);
151     int lowPos = normalize(low, globalLow, globalHigh, plotHeight);
152     int openPos = normalize(open, globalLow, globalHigh, plotHeight);
153     int closePos = normalize(close, globalLow, globalHigh, plotHeight);
154
155     normalizedCandlesticks.emplace_back(highPos, lowPos, openPos, closePos);
156 }
157
158 //generating the candlesticks text based graph
159 std::cout << "\n===== Candlestick Graph =====\\n";
160 for (int i = plotHeight; i >= 0; --i) {
161     double labelValue = globalLow + (globalHigh - globalLow) * i / plotHeight;
162     std::cout << std::setw(6) << std::fixed << std::setprecision(1) <<
labelValue << " |";
163
164     for (size_t j = 0; j < filteredData.size(); ++j) {
165         auto [highPos, lowPos, openPos, closePos] = normalizedCandlesticks[j];
166         bool isUpward = closePos > openPos; // True if green candlestick
167
168         std::string color = isUpward ? GREEN : RED;
169

```

```
170         if (i <= highPos && i > std::max(openPos, closePos)) {
171             std::cout << color << "    |    " << RESET;
172         } else if (i == openPos) {
173             std::cout << color << "    ---    " << RESET;
174         } else if (i == closePos) {
175             std::cout << color << "    ---    " << RESET;
176         } else if (i < std::min(openPos, closePos) && i >= lowPos) {
177             std::cout << color << "    |    " << RESET;
178         } else {
179             std::cout << "          ";
180         }
181     }
182     std::cout << "\n";
183 }
184
185 //drawing x-axis
186 std::cout << "          +";
187 for (size_t j = 0; j < filteredData.size(); ++j) {
188     std::cout << std::string(candlestickWidth, '-');
189 }
190 std::cout << "\n";
191
192 //drawing x-axis dates
193 std::cout << "          ";
194 for (const auto& row : filteredData) {
195     std::cout << std::setw(candlestickWidth) << row[0].substr(2, 5);
196 }
197 std::cout << "\n";
198 }
199 //used my teachers assistance in this code
```

CandlestickPlotter.h

```
1 //used my teachers assistance in this code
2 #ifndef CANDLESTICK_PLOTTER_H
3 #define CANDLESTICK_PLOTTER_H
4
5 #include <vector>
6 #include <string>
7
8 //the CandlestickPlotter class provides functionality to visually plot candlestick
graph
9 class CandlestickPlotter {
10 public:
11     //plots the OHLC data as a candlestick graph
12     void plotCandlesticks(const std::vector<std::vector<std::string>>& ohlcData);
13 };
14
15 #endif
16 //used my teachers assistance in this code
```

Country2020.cpp

```
1 //my code
2 #include "Country2020.h"
3 #include <fstream>
4 #include <sstream>
5 #include <iostream>
6 #include <algorithm>
7 #include <numeric>
8 #include <map>
9 #include "Candlestick.h"
10
11
12 //constructor
13 Country2020::Country2020(const std::string& filepath) {
14     loadData(filepath);
15 }
16
17 //loads data from file
18 void Country2020::loadData(const std::string& filepath) {
19     std::ifstream file(filepath);
20     if (!file.is_open()) {
21         std::cerr << "Error: Could not open file!" << std::endl;
22         return;
23     }
24
25     std::string line, header;
26     //reads the header line
27     getline(file, header);
28     std::istringstream headerStream(header);
29     std::string colName;
30
31     //extracting the country codes from the header
32     while (getline(headerStream, colName, ',')) {
33         if (colName != "utc_timestamp") {
34             countryCodes.push_back(colName.substr(0, 2));
35             countryData[colName.substr(0, 2)] = std::vector<double>();
36         }
37     }
38
39     //reading temperature data
40     while (getline(file, line)) {
41         std::istringstream lineStream(line);
42         std::string value;
43         int colIndex = 0;
44
45         while (getline(lineStream, value, ',')) {
46             if (colIndex > 0) {
47                 //skips the timestamp column
48                 countryData[countryCodes[colIndex - 1]].push_back(std::stod(value));
49             }
50             colIndex++;
51         }
52     }
53 }
54
55 //gets the country codes
56 const std::vector<std::string>& Country2020::getCountryCodes() const {
```

```

57     return countryCodes;
58 }
59
60
61 std::vector<Candlestick> Country2020::computeCandlestick(const std::string&
country) const {
62     //retrieves temperature data for the country
63     const auto& temperatures = countryData.at(country);
64     std::map<std::string, std::vector<double>> yearlyData;
65
66     //grouping the data by year
67     for (size_t i = 0; i < temperatures.size(); ++i) {
68         std::string year = std::to_string(1980 + (i / (365 * 24))); // Adjust
year calculation
69         yearlyData[year].push_back(temperatures[i]);
70     }
71
72     std::vector<Candlestick> candlesticks;
73     double previousClose = 0.0;
74
75     for (const auto& [year, temps] : yearlyData) {
76         //finds highest temperature of the year
77         double high = *std::max_element(temps.begin(), temps.end());
78         //finds lowest temperature of the year
79         double low = *std::min_element(temps.begin(), temps.end());
80         //calculates average temperature
81         double close = std::accumulate(temps.begin(), temps.end(), 0.0) /
temps.size();
82         //uses the previous years close as the current years open
83         double open = previousClose;
84
85         candlesticks.emplace_back(year, open, high, low, close);
86         previousClose = close;
87     }
88
89     return candlesticks;
90 }
91
92 std::tuple<double, double, double, double> Country2020::compute2020Values(const
std::string& countryCode) const {
93     //generates candlestick data for a country
94     const auto& candlesticks = this->computeCandlestick(countryCode);
95
96     double sumOpen = 0.0, sumHigh = 0.0, sumLow = 0.0, sumClose = 0.0;
97     //getting the number of candlesticks
98     int count = candlesticks.size();
99
100    for (const auto& candle : candlesticks) {
101        //accumulates open
102        sumOpen += candle.open;
103        //accumulates high
104        sumHigh += candle.high;
105        //accumulates low
106        sumLow += candle.low;
107        //accumulates close
108        sumClose += candle.close;
109    }
110
111    return {
112        //average open
113        sumOpen / count,

```

```
114 |         //average high
115 |         sumHigh / count,
116 |         //average low
117 |         sumLow / count,
118 |         //average close
119 |         sumClose / count
120 |     };
121 | }
122 | //my code
```

Country2020.h

```
1 //my code
2 #ifndef COUNTRY_2020_H
3 #define COUNTRY_2020_H
4
5 #include <string>
6 #include <vector>
7 #include <map>
8 #include "Candlestick.h"
9
10 //this Country2020 class handles temperature data for various countries allowing
    for predicting 2020 OHCL values
11 class Country2020 {
12 private:
13     //stores the temperature data for each country using country code.
14     std::map<std::string, std::vector<double>> countryData;
15     //list of country codes
16     std::vector<std::string> countryCodes;
17     //loads temperature data from a CSV file
18     void loadData(const std::string& filepath);
19
20 public:
21     //constructor that initializes the class by loading data from the provided file
22     Country2020(const std::string& filepath);
23     //gets the temperature data for a specific country code
24     const std::vector<double>& getTemperatureData(const std::string& country)
    const;
25     //returns a list of all available country codes
26     const std::vector<std::string>& getCountryCodes() const;
27     //computes candlestick data a countrys
28     std::vector<Candlestick> computeCandlestick(const std::string& country) const;
29     //computes 2020 OHCL values for any selected country
30     std::tuple<double, double, double, double> compute2020Values(const std::string&
    countryCode) const;
31
32 };
33
34 #endif
35 //my code
36
```

CountryTemperature.cpp

```
1 //my code
2 #include "CountryTemperature.h"
3 #include <fstream>
4 #include <sstream>
5 #include <iostream>
6 #include <algorithm>
7 #include <numeric>
8 #include <map>
9
10 CountryTemperature::CountryTemperature(const std::string& filepath) {
11     loadData(filepath);
12 }
13
14 //loading temperature data from the file and organizes it into country specific
15 void CountryTemperature::loadData(const std::string& filepath) {
16     std::ifstream file(filepath);
17     if (!file.is_open()) {
18         //file opening error
19         std::cerr << "Error: Could not open file!" << std::endl;
20         return;
21     }
22
23     std::string line, header;
24     //reading the header line
25     getline(file, header);
26     std::istringstream headerStream(header);
27     std::string colName;
28
29     //extracting country codes from the header
30     while (getline(headerStream, colName, ',')) {
31         //skips the timestamp
32         if (colName != "utc_timestamp") {
33             countryCodes.push_back(colName.substr(0, 2));
34             countryData[colName.substr(0, 2)] = std::vector<double>();
35         }
36     }
37
38     //reading the remaining lines containing temperature data
39     while (getline(file, line)) {
40         std::istringstream lineStream(line);
41         std::string value;
42         int colIndex = 0;
43
44         //distributing data into country vectors
45         while (getline(lineStream, value, ',')) {
46             if (colIndex > 0) {
47                 //skips the timestamp column
48                 countryData[countryCodes[colIndex - 1]].push_back(std::stod(value));
49             }
50             colIndex++;
51         }
52     }
53 }
54
55 //returns temperature data for a country
56 const std::vector<double>& CountryTemperature::getTemperatureData(const
std::string& country) const {
```

```
57 //accesses the data using the country code
58 return countryData.at(country);
59 }
60
61 //returns the list of available countrirs
62 const std::vector<std::string>& CountryTemperature::getCountryCodes() const {
63     return countryCodes; // Provide a reference to the country codes vector
64 }
65
66 //computing candlestick data for a given country's yearly temperature data
67 std::vector<Candlestick> CountryTemperature::computeCandlestick(const std::string&
country) const {
68     const auto& temperatures = countryData.at(country);
69     std::map<std::string, std::vector<double>> yearlyData;
70
71     //grouping temperature data by years
72     for (size_t i = 0; i < temperatures.size(); ++i) {
73         //calculates year based on index
74         std::string year = std::to_string(1980 + (i / (365 * 24)));
75         yearlyData[year].push_back(temperatures[i]);
76     }
77
78     std::vector<Candlestick> candlesticks;
79     double previousClose = 0.0;
80
81     //creates candlesticks for each year till 2019
82     for (const auto& [year, temps] : yearlyData) {
83         if (std::stoi(year) > 2019) {
84             break;
85         }
86         //finds the highest and lowest temperature
87         double high = *std::max_element(temps.begin(), temps.end());
88         double low = *std::min_element(temps.begin(), temps.end());
89         //calculates average
90         double close = std::accumulate(temps.begin(), temps.end(), 0.0) /
temps.size();
91         //uses the previous years close as the current years open
92         double open = previousClose;
93
94         //adds candlestick for the year
95         candlesticks.emplace_back(year, open, high, low, close);
96         //updates the previous close for the next iteration
97         previousClose = close;
98     }
99
100     return candlesticks;
101 }
102 //my code
```

CountryTemperature.h

```
1 //my code
2 #ifndef COUNTRY_TEMPERATURE_H
3 #define COUNTRY_TEMPERATURE_H
4
5 #include <string>
6 #include <vector>
7 #include <map>
8 #include "Candlestick.h"
9
10 //this CountryTemperature class handles temperature data for various countries
    allowing for retrieval and candlestick style analysis
11 class CountryTemperature {
12 private:
13     //stores the temperature data for each country
14     std::map<std::string, std::vector<double>> countryData;
15     //list of country codes
16     std::vector<std::string> countryCodes;
17     //loads temperature data from a CSV file
18     void loadData(const std::string& filepath);
19
20 public:
21     //constructor that initializes the class by loading data from the provided file
22     CountryTemperature(const std::string& filepath);
23     //gets the temperature data for a specific country code
24     const std::vector<double>& getTemperatureData(const std::string& country)
const;
25     //returns a list of all available country codes
26     const std::vector<std::string>& getCountryCodes() const;
27     //computes candlestick data a countrys
28     std::vector<Candlestick> computeCandlestick(const std::string& country) const;
29
30 };
31
32 #endif
33 //my code
```

main.cpp

```
1 //my code
2 #include <iostream>
3 #include "CountryTemperature.h"
4 #include "Country2020.h"
5 #include "Menu.h"
6
7 int main() {
8     //specifying the path to the weather data file
9     std::string filepath = "weather_data.csv";
10
11     //loading temperature data and initializing the menu system
12     CountryTemperature tempData(filepath);
13     Country2020 tempoData(filepath);
14     Menu menu(tempData, tempoData);
15
16     //main application loop
17     while (true) {
18         //displaying the menu options
19         std::cout << "===== Menu =====" <<
std::endl;
20         std::cout << "1. OCHL Values & Candlesticks" << std::endl;
21         std::cout << "2. Predicting OCHL Values for 2020 " << std::endl;
22         std::cout << "0. Exit" << std::endl;
23         std::cout << "Select one option: ";
24
25         int choice;
26         std::cin >> choice;
27
28         //handles user input
29         switch (choice) {
30             case 1:
31                 //shows OHCL table and candlestick data
32                 menu.show();
33                 break;
34             case 2:
35                 //shows predicted 2020 OHCL values
36                 menu.showed();
37                 break;
38             case 0:
39                 //exit the program
40                 std::cout << " Goodbye :( " << std::endl;
41                 return 0;
42             default:
43                 //handles invalid input
44                 std::cout << "Invalid choice. Please try again." << std::endl;
45         }
46     }
47 }
48 //my code
```

Menu.cpp

```

1 //my code
2 #include "Menu.h"
3 #include <iostream>
4 #include <iomanip>
5
6 // Define the dummy objects
7 CountryTemperature Menu::dummyCountryTemperature("weather_data.csv");
8 Country2020 Menu::dummyCountry2020("weather_data.csv");
9
10
11 //displaying the list of countries for selection
12 void Menu::displayCountryList() {
13     const auto& countries = tempData.getCountryCodes();
14     std::cout << "===== Countries =====" << std::endl;
15     for (size_t i = 0; i < countries.size(); i++) {
16         //listing countries with numbering
17         std::cout << i + 1 << ". " << countries[i] << std::endl;
18     }
19     //option to return to the previous menu
20     std::cout << "0. Back" << std::endl;
21 }
22
23 //displaying OHLC data for the selected country and generatting a candlestick
graph
24 void Menu::showCountryTemperature(const std::string& countryCode) {
25     auto candlesticks = tempData.computeCandlestick(countryCode);
26
27     //displaing yearly OHLC data in a table format
28     std::cout << "\nYearly OHLC Data for " << countryCode << ":" << std::endl;
29     std::cout << std::setw(10) << "Year" << " "
30         << std::setw(10) << "Open" << " "
31         << std::setw(10) << "High" << " "
32         << std::setw(10) << "Low" << " "
33         << std::setw(10) << "Close" << std::endl;
34
35     for (const auto& candle : candlesticks) {
36         std::cout << std::setw(10) << candle.date << " "
37             << std::setw(10) << candle.open << " "
38             << std::setw(10) << candle.high << " "
39             << std::setw(10) << candle.low << " "
40             //displaying each candlesticks data
41             << std::setw(10) << candle.close << std::endl;
42     }
43
44     //preparing data for the plotCandlesticks function
45     std::vector<std::vector<std::string>> formattedCandlesticks;
46     for (const auto& candle : candlesticks) {
47         formattedCandlesticks.push_back({
48             candle.date,
49             std::to_string(candle.open),
50             std::to_string(candle.high),
51             std::to_string(candle.low),
52             std::to_string(candle.close)
53         });
54     }
55
56     //generating and displaying the candlestick graph

```

```

57     std::cout << "\nCandlestick Graph:" << std::endl;
58     plotter.plotCandlesticks(formattedCandlesticks);
59 }
60
61 //handlling the main menu interaction for selecting and displaying country data
62 void Menu::show() {
63     while (true) {
64         //showing the list of countries
65         displayCountryList();
66         std::cout << "Select a country by number : ";
67         int choice;
68         std::cin >> choice;
69
70         //exits the menu
71         if (choice == 0) {
72             break;
73         }
74
75         const auto& countries = tempData.getCountryCodes();
76         if (choice > 0 && static_cast<size_t>(choice) <= countries.size()) {
77             //displaying temperature data for the selected country
78             showCountryTemperature(countries[choice - 1]);
79         } else {
80             //handle invalid input
81             std::cerr << "Invalid choice! Please try again." << std::endl;
82         }
83     }
84 }
85
86 void Menu::showCountry2020(const std::string& countryCode) {
87     auto [avgOpen, avgHigh, avgLow, avgClose] =
88     tempoData.compute2020Values(countryCode);
89     std::cout << "\n2020 OHLC Values (Computed Averages):\n";
90     std::cout << "Open: " << avgOpen << "\n";
91     std::cout << "High: " << avgHigh << "\n";
92     std::cout << "Low: " << avgLow << "\n";
93     std::cout << "Close: " << avgClose << "\n";
94
95     //plots the bar graph
96     plotBarGraph(avgOpen, avgHigh, avgLow, avgClose);
97 }
98
99 void Menu::showed() {
100     int choice;
101     do {
102         //displays the list of countries for users to select
103         displayCountryList();
104         std::cout << "Select a country by number : ";
105         std::cin >> choice;
106
107         //checking if the choice is valid and within the range of available
108         countries
109         if (choice > 0 && choice <= tempoData.getCountryCodes().size()) {
110             std::string countryCode = tempoData.getCountryCodes()[choice - 1];
111             //displaying the selcted countriies info
112             showCountry2020(countryCode);
113         }
114         //exit
115     } while (choice != 0);
116 }

```

```
115
116 void Menu::plotBarGraph(double open, double high, double low, double close) {
117     //height of the graph
118     const int graphHeight = 10;
119     //width of each bar
120     const int barWidth = 5;
121
122     //determining the range of values to normalize the graph
123     double minValue = std::min({open, high, low, close});
124     double maxValue = std::max({open, high, low, close});
125     double range = maxValue - minValue;
126
127     //normalizing the OHLC values
128     int openHeight = static_cast<int>((open - minValue) / range * graphHeight);
129     int highHeight = static_cast<int>((high - minValue) / range * graphHeight);
130     int lowHeight = static_cast<int>((low - minValue) / range * graphHeight);
131     int closeHeight = static_cast<int>((close - minValue) / range * graphHeight);
132
133     //displaying the the OHLC values and prepare for graph visualization
134     std::cout << "\nPredicted 2020 OHLC Values:\n";
135     std::cout << "Open: " << open << ", High: " << high << ", Low: " << low << ",
Close: " << close << "\n";
136     std::cout << "Bar Graph:\n";
137
138     //y axis labels
139     std::cout << std::fixed << std::setprecision(2);
140     for (int i = graphHeight; i >= 0; --i) {
141         double yValue = minValue + (range * i / graphHeight);
142         std::cout << std::setw(6) << yValue << " |";
143
144         //renders a green color open bar
145         std::cout << "\033[32m";
146         std::cout << "|";
147         for (int j = 0; j < barWidth; ++j) {
148             if (i <= openHeight) {
149                 std::cout << "0";
150             } else {
151                 std::cout << " ";
152             }
153         }
154         //resets color
155         std::cout << "\033[0m";
156         std::cout << "|";
157
158         //renders a red color high bar
159         std::cout << "\033[31m";
160         std::cout << "|";
161         for (int j = 0; j < barWidth; ++j) {
162             if (i <= highHeight) {
163                 std::cout << "H";
164             } else {
165                 std::cout << " ";
166             }
167         }
168         //resets color
169         std::cout << "\033[0m";
170         std::cout << "|";
171
172         //renders a blue color low bar
173         std::cout << "\033[34m";
```

```
174     std::cout << "|";
175     for (int j = 0; j < barWidth; ++j) {
176         if (i <= lowHeight) {
177             std::cout << "L";
178         } else {
179             std::cout << " ";
180         }
181     }
182     //resets color
183     std::cout << "\033[0m";
184     std::cout << "|";
185
186     //renders a yellow color close bar
187     std::cout << "\033[33m";
188     std::cout << "|";
189     for (int j = 0; j < barWidth; ++j) {
190         if (i <= closeHeight) {
191             std::cout << "C";
192         } else {
193             std::cout << " ";
194         }
195     }
196     //reseets color
197     std::cout << "\033[0m";
198     std::cout << "|\n";
199 }
200
201 //drawing the x axis
202 std::cout << "      +";
203 for (int i = 0; i < 4; ++i) {
204     std::cout << std::string(barWidth + 2, '-');
205 }
206 std::cout << "+\n";
207
208 //labeling the bars
209 std::cout << "          Open      High      Low      Close\n";
210 }
211 //my code
```

Menu.h

```
1 //my code
2 #ifndef MENU_H
3 #define MENU_H
4
5 #include "CountryTemperature.h"
6 #include "Country2020.h"
7 #include "CandlestickPlotter.h"
8
9 //this Menu class provides a user interface for interacting with temperature data
  allowing users to view OHLC values and generate candlestick graph
10 class Menu {
11 private:
12     CountryTemperature& tempData;
13     Country2020& tempoData;
14
15     //dummy objects for initialization
16     static CountryTemperature dummyCountryTemperature;
17     static Country2020 dummyCountry2020;
18
19     //instance of CandlestickPlotter for visualizing data
20     CandlestickPlotter plotter;
21
22     //displaying a list of countries for user selection
23     void displayCountryList();
24     //displaying OHLC values and candlestick data for a specific country
25     void showCountryTemperature(const std::string& countryCode);
26     //displaying predicted 2020 OHLC values for a specific country
27     void showCountry2020(const std::string& countryCode);
28
29 public:
30     //constructor for CountryTemperature
31     Menu(CountryTemperature& data) : tempData(data), tempoData(dummyCountry2020) {}
32     //constructor for Country2020
33     Menu(Country2020& data) : tempData(dummyCountryTemperature), tempoData(data) {}
34     //new constructor for both CountryTemperature and Country2020
35     Menu(CountryTemperature& data, Country2020& data2020) : tempData(data),
tempData(data2020) {}
36
37     //declarations
38     void show();
39     void showed();
40     void plotBarGraph(double open, double high, double low, double close);
41
42 };
43
44 #endif
45 //my code
```